

PS002

Purpose-Based Consent Recording and Withdrawal

JOEL KURIAN ABRAHAM
joelka.imca253o@saintgits.org

KELVIAH C BINO
kcb.imca253o@saintgits.org

AIORI Hackathon 3

October 13, 2026

Problem Description

Clear outline of the problem being addressed. Use bullet points for key issues.

Consent records are often scattered across forms, applications, and service databases. A decision may be difficult to connect to the exact purpose and notice that the person saw, and a withdrawal may not reach every service that relies on the original grant. PS-002 proposes a small prototype that keeps purpose-specific decisions traceable and makes current status available at the point of use.

- A broad, bundled choice can obscure which individual purpose a person accepted.
- A changed notice or purpose may leave an old grant ambiguous.
- A relying service may process data without checking whether a later refusal or withdrawal exists.
- Overwriting the previous value removes evidence of when and how consent changed.

Prototype objective: capture each choice as a versioned event, evaluate the current status for a subject-purpose pair, and fail closed when that status cannot be established.

Background

It should capture the background necessary to understand the problem and the proposed optimization

Consent is meaningful only when a person can understand the stated purpose and make a clear choice. For a consent-based workflow, the system needs to keep the purpose, scope, notice version, actor, and time associated with the decision. A later decision must be represented without deleting the earlier record.

The prototype separates three concerns: a purpose and notice catalog, an append-only event history, and a status evaluator. The relying service asks for a decision tied to one purpose rather than receiving an unrestricted copy of the complete consent record.

A withdrawal changes the status used for future checks and triggers a downstream notification. It does not itself define the handling of processing that occurred earlier or every copy already held by another system; those are policy and operational decisions that must be specified for a real deployment [1].

Literature & RFC Review

Do a detailed study of existing proposals, studies and standards. But only mention the most relevant ones

[1] Regulation (EU) 2016/679 (GDPR), especially Articles 7 and 5. Article 7 addresses conditions for consent, including the ability to withdraw; the regulation states that withdrawal does not affect the lawfulness of processing based on consent before withdrawal. This supports a versioned record and a clear effective time. The prototype does not determine whether consent is the appropriate legal basis for any particular use.

[2] NIST Privacy Framework, Version 1.0. Its risk-management approach informs the design choices to minimize subject data, define access, and monitor downstream handling. It is a voluntary framework, not a binding legal rule.

[3] RFC 9700, Best Current Practice for OAuth 2.0 Security. Where OAuth protects status-check APIs, its current security guidance is relevant to service authentication, token handling, and secure API access.

Design implication: record the exact notice and purpose version, authenticate relying services, minimize the response, and preserve an auditable sequence of events.

Proposed Solution

Detailed description of the overall solution.

PS-002 provides a subject-facing consent page, a purpose and notice catalog, a consent API, an append-only event store, and a status-check endpoint for relying services. Every purpose has a stable identifier, clear description, scope, and notice version. A person chooses grant or refuse separately for each purpose; withdrawal is available for an active grant.

When a service is ready to process data, it submits the pseudonymous subject reference, purpose identifier, and requested scope. The evaluator checks the latest valid event and relevant version policy, then returns an allow or deny decision with an effective time and correlation reference. An unavailable or ambiguous status results in a deny or indeterminate response, never an assumed grant.

A withdrawal appends a new event and publishes a notification for connected systems. Retries use stable event identifiers so downstream handlers can avoid duplicate effects. Access checks are separately audited.

Optimization Proposed

List of optimizations and enhancements. Use a table to compare before and after scenarios.

The prototype replaces informal or disconnected consent handling with purpose-scoped records and a consistent lookup path.

AREA	BEFORE	PROPOSED AFTER
Choice	One general setting may cover multiple uses.	Separate action for each named purpose and scope.
Evidence	Current value may overwrite earlier choices.	Append-only events preserve grant, refusal, and withdrawal history.
Notice changes	A decision may not identify the text shown.	Record the notice version and purpose version with the event.
Use-time check	Service may rely on a cached or assumed permission.	Check current status for each subject-purpose pair; fail closed.
Withdrawal	Update may not reach connected services.	Append a withdrawal event and deliver a retryable notification.

Expected improvement: clearer individual choices, more reliable status decisions, and a traceable basis for review.

Solution Architecture & Design

Visual: Flowchart or diagram showing the optimization process.

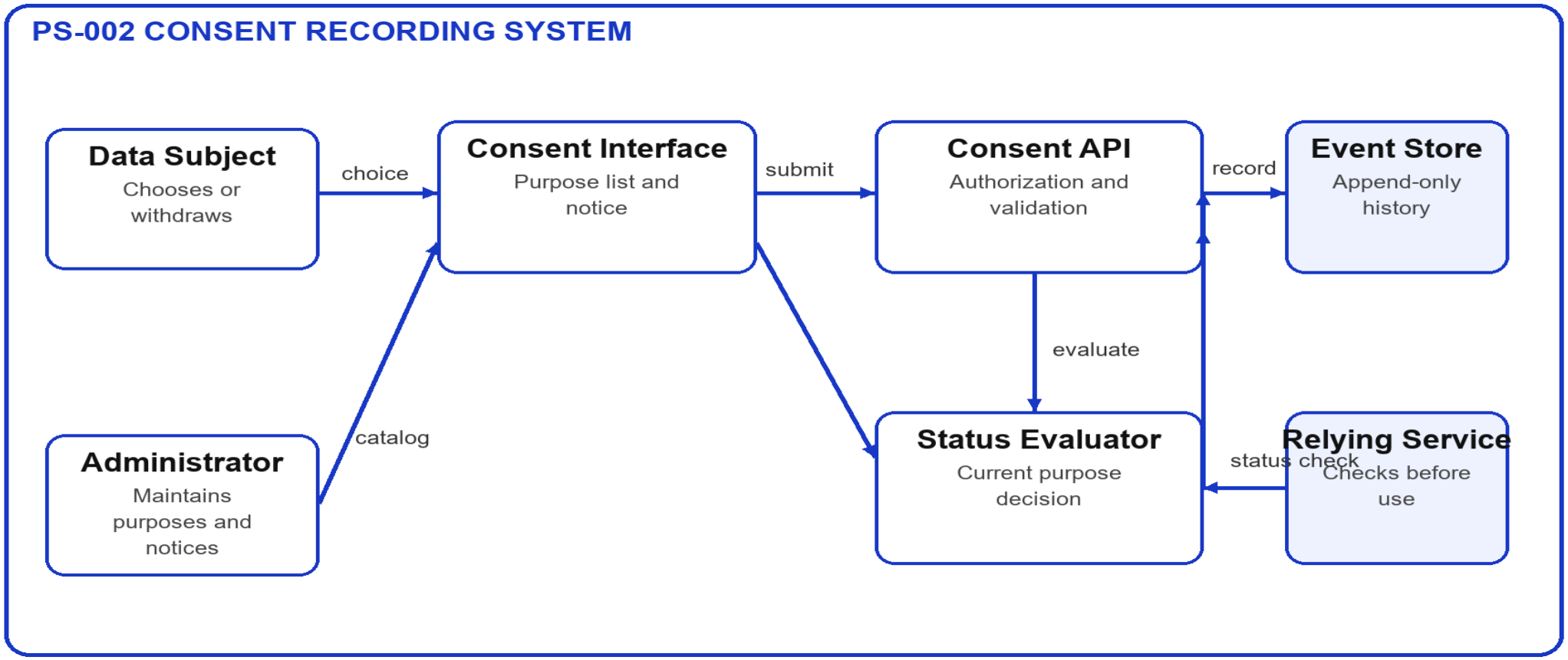


Figure: System Architecture UML Diagram

The event store is the durable record. The status evaluator derives the current decision from purpose-specific events; the relying service receives only the decision and its trace reference.

Data Flow Diagram

Overview of the architecture. Include the flow chart diagram created earlier. Visual: Diagrams or architecture visuals.

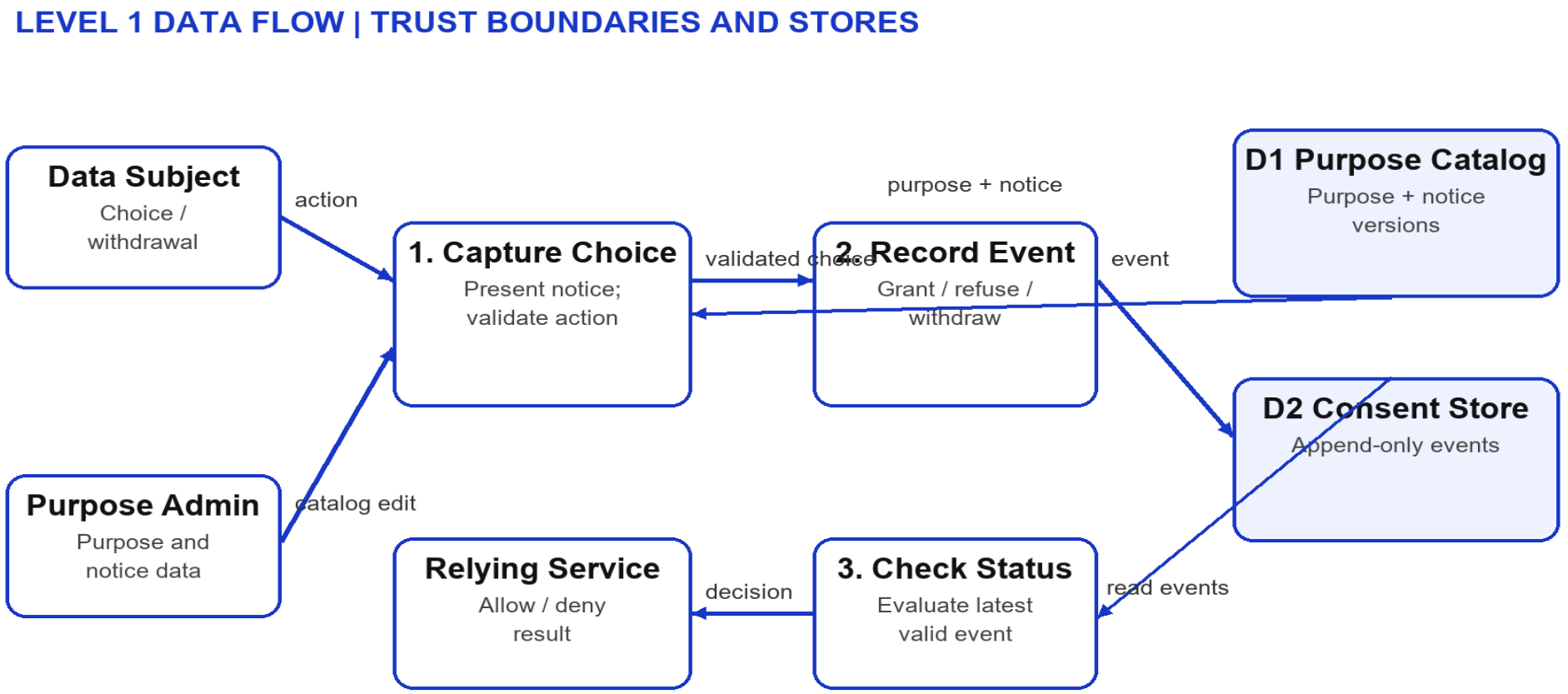


Figure: Data Flow Diagram (DFD)

The trust boundary sits at the consent API. Requests are authenticated and scoped; the event store is append-only; downstream withdrawal delivery is retryable and deduplicated.

Deployment Timeline

Clearly define the deployment milestones and methodology.

Use short prototype iterations. Validate each stage with the consent lifecycle scenarios before connecting a relying service.

MILESTONE	WORK	EXIT CHECK
1. Define purposes	Write purpose identifiers, scope, notice text, and version policy.	Each purpose is clear and independently selectable.
2. Build consent experience	Implement grant, refuse, and withdrawal actions.	Keyboard-accessible; no preselected grant; confirmation is shown.
3. Store events	Add event IDs, timestamps, actor, notice version, and idempotency.	Prior events remain intact after a new choice.
4. Evaluate status	Implement current status and fail-closed behavior.	All grant, refusal, withdrawal, and stale-version scenarios pass.
5. Integrate and review	Connect one relying service; add audit and retry monitoring.	Service decisions are scoped, traceable, and withdrawal is reconciled.

Methodology: build one purpose end to end, test it, then add the remaining purposes and integrations.

Bibliography I

[1] European Parliament and Council of the European Union. Regulation (EU) 2016/679 (General Data Protection Regulation), Articles 5 and 7. Official Journal of the European Union, 2016. https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng/

[2] National Institute of Standards and Technology. NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0. January 16, 2020. https://www.nist.gov/privacy-framework

[3] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett. RFC 9700: Best Current Practice for OAuth 2.0 Security. IETF, January 2025. https://www.rfc-editor.org/rfc/rfc9700.html

Thank You For Your Attention!